

Solutions for YOLO Multi-Model Deployment Issues

Comprehensive Architectural Blueprint to Migrate from Two Model Pipelines into a Single Unified Multi-Class Network

Executive Summary: Running two separate .pt models simultaneously causes severe performance degradation due to redundant preprocessing, double inference forward passes, and disjointed Non-Maximum Suppression (NMS) execution. The architectural solution is to merge the independent datasets and retrain a **single multi-class model**. When edge resource constraints require temporary pipeline adjustments, strict software wrapper layers must handle shared array preprocessing, asynchronous execution threads, or consolidated NMS tensors.

Identified Problem (Multi-.pt Engine)	Architectural Solution Blueprint
1. Double Preprocessing Each network runs isolated resize and image normalization routines independently, duplicating memory modifications.	Single-Pass Tensor Streaming: Manually wrap inputs using NumPy/OpenCV. Resize, normalize, and swap channels <i>once</i> to prepare a unified CHW tensor matrix, then stream that exact pointer simultaneously into both models to eliminate compute overhead.
2. Double Inference Latency Every graphic frame triggers two serial network graphs, creating bottlenecks that limit processing frame-rates.	Asynchronous Thread Concurrency: Use Python's threading module to dispatch frame tensors to separate GPU stream cues concurrently, or port pipelines into a unified model containing multiple prediction outputs.
3. High Resource Overhead Loading multiple .pt neural branches concurrently triggers massive VRAM footprint requirements.	Quantized Edge Export: Export models to lightweight, localized runtime architectures like ONNX Runtime or TensorRT . Compress heavy fp32 architectures down to dynamic int8 formats to drop system memory utilization.
4. Fragmented NMS Step Independent NMS calls trigger overlapping bounding box arrays across decoupled model spaces.	Unified Bounding Box Tensors: Concatenate output box coordinate tensors, confidence vectors, and target indices from both engines into a single contiguous array, then execute <i>one</i> final global NMS filter pass.
5. Complex Pipeline Logic Writing code to align, merge, and clean multi-engine predictions increases maintenance overhead.	Standardized Framework Packaging: Wrap your entire prediction stack inside a unified object-oriented Python class structure. Ensure all model pipelines ingest matching array shapes and emit uniform dictionary structures.

6. Deployment Difficulty Managing separate weights files in cloud production endpoints increases connection routing latency.	Unified Endpoint Containerization: Package all engine checkpoints inside an isolated microservice using frameworks like FastAPI, exposing a single secure endpoint to optimize container scale paths.
7. Slower Scaling Paths Adding additional target objects (like cracks or debris) forces adding extra weight tracks, degrading performance.	Consolidated Multi-Class Retraining: Merge distinct labels into one global coordinate system mapping layout (e.g., class 0: pothole, class 1: garbage, class 2: puddle) and train a single multi-class network.

Production Implementation: Single-Pass Preprocessing & Unified Inferences

```
import cv2
import numpy as np
from ultralytics import YOLO

class OptimizedDualModelPipeline:
    def __init__(self, model_a_path, model_b_path):
        # Load weights into memory array matrix spaces
        self.model_a = YOLO(model_a_path)
        self.model_b = YOLO(model_b_path)

    def preprocess_once(self, frame_bgr, target_size=640):
        # Ingest raw camera array and apply standard geometric normalization ONCE
        resized = cv2.resize(frame_bgr, (target_size, target_size))
        rgb = cv2.cvtColor(resized, cv2.COLOR_BGR2RGB)
        tensor = rgb.astype(np.float32) / 255.0 # Scale pixel float matrices
        tensor = np.transpose(tensor, (2, 0, 1)) # Convert HWC layout to CHW
        tensor = np.expand_dims(tensor, axis=0) # Create batch frame space [1, 3, 640, 640]
        return tensor

    def predict_unified(self, frame_matrix):
        # 1. Share the exact same preprocessed tensor across both network tracks
        shared_tensor = self.preprocess_once(frame_matrix)

        # 2. Run prediction calls directly on the prepared raw matrix arrays
        results_a = self.model_a.predict(source=shared_tensor, verbose=False)
        results_b = self.model_b.predict(source=shared_tensor, verbose=False)

        return results_a, results_b
```

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.